

Sliding Window definition:

Sliding Window is an optimization technique used to solve problems involving **contiguous (continuous) subarrays or substrings** by maintaining a window of elements and moving it across the data instead of recomputing results for every possible range.

Types:

- 1) Fixed Size Sliding Window
- 2) Variable Size Sliding Window

1. MAXIMUM EMPLOYEES IN A TEAM

You are given skill levels of N employees.
A team is valid if the difference between the highest and lowest skill in the team is at most 5.
Find the maximum number of employees that can be in a valid team.

Logic (Sliding Window)

1. Sort the array

1 3 5 7 8 9

2. Use two pointers (window) to keep $\max - \min \leq 5$

1 3 5 7 8 9
L R

Window: 3 5 7 8 (Size = 4)

Idea: Expand the window with R, shrink with L when $(\max - \min > 5)$

Example:

Input: 6

1 3 5 7 8 9

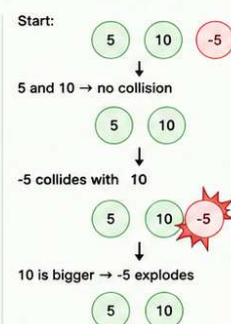
Output: 4

Explanation: One possible team is
3 5 7 8 ($8 - 3 = 5$)

2. ASTEROID COLLISION (LC-735)

You are given an array **asteroids** of integers.
Each integer represents an asteroid.
Positive value → moving right.
Negative value → moving left.
When two asteroids collide, the smaller one explodes. If equal, both explode.
Return the state of asteroids after all collisions.

Logic (Stack)



Example:

Input: [5, 10, -5]

Output: [5, 10]

Explanation:
-5 moves left and collides with 10.
10 is bigger, so -5 explodes.
Remaining: [5, 10]

Idea: Use stack to keep asteroids.
Check collision with top of stack.

3. TRAIN TRAVEL TIME

There are N+1 stations. Between every pair of consecutive stations, two trains are available: Train A and Train B.
You must travel using exactly one continuous segment with Train A (of length K).
All remaining segments must be traveled using Train B.
Find the minimum total travel time.

Logic (Sliding Window)

1. First, compute total time using Train B only

Total B = $3 + 3 + 1 + 2 + 2 = 11$

2. We will replace any window of size K with Train A and see the minimum.

Compute $(A[i] - B[i])$ for each index

0 1 2 3 4
-1 -2 1 2 1

Now find maximum sum of any window of size K = 2

(0-1) (1-2) (2-3) (3-4)
-3 -1 3 3

Minimum Total = Total B - Max(window sum)
= $11 - 3 = 10$

Example:

N = 5

Train A: 2 1 2 4 3

Train B: 3 3 1 2 2

K = 2

Output: 10

Idea: Use Fixed Size Sliding Window on $(A[i] - B[i])$

4. FACTORY PRODUCTION

A factory produces N products in a line.
0 → defective product
1 → good product
You can repair at most K defective products (change 0 to 1).
Find the maximum number of consecutive good products that can be obtained.

Logic (Sliding Window)

Use window and count zeros.
At most K zeros are allowed.
Expand R, if $\text{zeros} > K \rightarrow$ move L

Array:

1 1 0 0 1 1 1 0 1 1
0 1 2 3 4 5 6 7 8 9

Window

L R

Best window after repairs:

1 1 1 1 1 1 1

Length = 7

Example:

Input:

N = 10, K = 2

Arr = [1, 1, 0, 0, 1, 1, 1, 0, 1, 1]

Output: 7

Explanation:
Repair the 0's at index 2 and 3.
Longest consecutive 1's = 7

Idea: This is Max Consecutive Ones III

SLIDING WINDOW – THE COMPLETE GUIDE

1. WHAT IS SLIDING WINDOW?

It is a technique where we maintain a “**window**” (a contiguous part) of the array/sequence and slide it from left to right to solve problems efficiently.

Example Array

4	6	8	9	10	12	17	24
0	1	2	3	4	5	6	7

2. WHY DO WE USE IT?

- Brute force checks all possible subarrays $\rightarrow O(n^2)$
- Sliding Window checks each element at most twice (one time by R, one time by L) $\rightarrow O(n)$
- Very useful for problems on “contiguous” ranges.

3. TYPES OF SLIDING WINDOW

Fixed Size Window

Window size is fixed.
We move both pointers by 1 step.

Example: Max sum of k consecutive elements.

Variable Size Window

Window size can grow or shrink depending on condition.

Example: Longest subarray with $\max - \min \leq K$.

4. HOW DOES IT WORK? (VARIABLE SIZE WINDOW)

Let's solve: **Largest group** where $\max - \min \leq 5$

Step 1: Start
Window = [4]
Valid (0)

Step 2: R moves
Window = [4, 6]
 $\max - \min = 2$ ✓

Step 3: R moves
Window = [4, 6, 8]
 $\max - \min = 4$ ✓

Step 4: R moves
Window = [4, 6, 8, 9]
 $\max - \min = 5$ ✓

Step 5: R moves
Window = [4, 6, 8, 9, 10]
 $\max - \min = 6$ ✗ (Invalid)

Step 6: Move L
Window = [6, 8, 9, 10]
 $\max - \min = 4$ ✓

Step 7: R moves
Window = [6, 8, 9, 10, 12]
 $\max - \min = 6$ ✗ (Invalid)

Step 8: Move L
Window = [8, 9, 10, 12]
 $\max - \min = 4$ ✓

We keep moving R to expand.
If invalid, we move L to shrink until valid again.
At each step, update the answer (window size).

Maximum window size
here is 4 (e.g., [4,6,8,9] or [8,9,10,12])

5. GENERAL PATTERN (ALGORITHM)

```
left = 0
answer = 0

for right in range(0, n):
    add arr[right] to window data    # Expand window

    while window is invalid:        # Shrink window
        remove arr[left] from window
        left += 1

    # Now window is valid
    update answer using current window

return answer
```

6. WHY DOES IT WORK? (INTUITION)

Think of Two Pointers (L and R)

- R always moves forward to explore new elements.
- When condition breaks, we move L forward to remove elements from the left until it becomes valid.
- Both pointers only move forward, never backward.

Why is it $O(n)$?

- R moves $\rightarrow$
- L moves $\rightarrow$
- Each element is visited by R at most once.
 - Each element is removed by L at most once.
 - Total operations $\leq 2n \rightarrow O(n)$

7. WHEN TO USE?

Whenever the problem asks for maximum/minimum/longest/shortest subarray or substring that satisfies a condition.

Keywords: longest, shortest, maximum, minimum, subarray, substring, continuous, at most, at least, equals to, distinct characters, $\sum \leq K$, etc.

SUMMARY

Expand R

Check Condition

If Invalid Move L

Update Answer

Repeat

8. QUICK CHECK EXAMPLE

Array: [2, 1, 5, 2, 3, 2] $K = 4$
Find length of longest subarray with $\sum \leq K$

0	1	2	3	4	5
2	1	5	2	3	2

Answer: 3
(Subarray [1, 5, 2]
or [2, 3, 2])

Consider a Train

Initially

$[1\ 2\ 3\ 4\ 5\ 6\ 7\ 8]$

$[1\ 2\ 3] \mid 4\ 5\ 6\ 7\ 8$

$1[2\ 3\ 4] \mid 5\ 6\ 7\ 8$

$1\ 2[3\ 4\ 5] \mid 6\ 7\ 8$

$1\ 2\ 3[4\ 5\ 6] \mid 7\ 8$

Maximum

A = 1

Condition

1)

2)

Set S

On

Maximum Employees Team

Team Contains

4)

0	1	2	3	4	5	6	7
4	6	8	9	10	12	17	24

Employees

$$\downarrow$$

$$[A]$$

2) They wanted to make team but the skill should be less than (or) equal to 5.

4

$$4 - 4 = 0$$
$$\boxed{\text{Size} = 1}$$

4, 6

$$6 - 4 = 2 \text{ (valid)}$$

Size = 2

4, 6, 8

$$\Rightarrow 8 - 4 = 4 \text{ (valid)}$$

Singe = 3

G4

4, 6, 8, 9

$$9-4=5 \text{ (valid)}$$
$$\sqrt[3]{S_{\text{Seje}}} = 4$$

675

④, 6, 8, 9, 10

$10 - 4 \Rightarrow 6$ (not valid)

B7C

6, 8, 9, 10

10-6 $\Rightarrow$ 4 (valid)

$$S_{yi} = 4$$

67

8, 9, 10, 12

$12 - 6 \Rightarrow 6$
(Not valid)

2.

678
 8, 9, 10, 12, 17
 vector 12-8 $\Rightarrow$ 4 (valid)
 length = 4
 List: [8, 9, 10, 12, 17]

↳

□

✓

✓

↓
✗

□

—

Question 1: Maximum Employees in a Team

Problem Statement

A company is forming a project team from its employees.

Each employee has a **skill level**, represented by an integer.

A team is considered **compatible** if the difference between the **highest** and **lowest** skill level in the team is **at most 5**.

Your task is to determine the **maximum number of employees** that can be selected to form a compatible team.

Input Format

- The first line contains an integer **N**, representing the number of employees.
 - The second line contains **N** space-separated integers representing the skill levels of the employees.
-

Output Format

Print a single integer representing the maximum number of employees that can form a compatible team.

Input: 8

Array=[4,17,8,24,6,10,12,9]

Output: 4


```
public class MaximumEmployeeTeam {  
  
    // Function to find the maximum number of employees  
    public static int findMaxEmployees(int[] skills) 1 usage  
    {  
        int left=0;  
        int maxTeam=0;  
        Arrays.sort(skills);  
  
        for(int right=0;right<skills.length;right++)  
        {  
            if(skills[right]-skills[left]>5)  
            {  
                left++;  
            }  
            int currentSize=right-left+1;  
            maxTeam=Math.max(currentSize,maxTeam);  
        }  
        return maxTeam;  
    }  
}
```